# Linux Commands

**ps (Process Status)**
**Purpose**
The ps command is used to display information about the currently running processes on a system.
It shows details such as **process ID (PID)**, **terminal association**, **CPU usage**, **memory usage**, and **the command that started the process**.

---

**Basic Syntax**
ps [options]
If used without any options, ps displays only the processes running in the **current terminal session**.
**Common Examples**
**1. Basic Usage**
ps
**Output Example:**

```
 PID TTY          TIME CMD
1234 pts/0    00:00:00 bash
1456 pts/0    00:00:01 ps
```

**Explanation:**

- **PID:** Process ID — unique number assigned to each process
- **TTY:** Terminal type (e.g., pts/0 → pseudo terminal)
- **TIME:** Total CPU time used by the process
- **CMD:** Command that started the process

2.ps -e ,ps -A

- Shows all processes running on the system (not just your terminal).

### 3. View Process Tree

- **`ps -ef --forest`**
- Displays processes in a hierarchical tree structure showing parent-child relationships.

### 4. Display Processes for a Specific User

- **`ps -u username`**
- **Lists all processes owned by the given user.**
- ---
- **8. Show Process IDs Only**
- **`ps -C processname -o pid=`**
- **Example:**
- **`ps -C firefox -o pid=`**
- **Outputs only the PID(s) of processes named "firefox".**

# Command: `top`

**The top command displays dynamic, real-time information about system processes, including:**

- **Process IDs (PIDs)**
- **CPU and memory usage**
- **User ownership**
- **System uptime**
- **Load average**
  **and much more.**

**It's essentially a live dashboard for your system.**

---

**Basic Syntax**
**top [options]**
**or simply:**
**top**

---

**Example:**
**top**
**Typical Output:**
**top - 10:41:07 up 2 days, 2:21, 2 users, load average: 0.23, 0.14, 0.09**

| | |
|---|---|
| *10:41:07* | *Current system time* |
| *up 2 days, 2:21* | *System uptime* |
| *2 users* | *Number of logged-in users* |
| *load average* | *Average number of processes waiting for CPU over 1, 5, and 15 minutes* |

*Load average of 1.00 on a single-core CPU means fully utilized CPU.*
*Values above 1.00 indicate CPU congestion.*

**Tasks: 198 total, 1 running, 197 sleeping, 0 stopped, 0 zombie**

| *Field* | *Meaning* |
|---|---|
| *total* | *Total number of processes* |
| *running* | *Currently executing processes* |
| *sleeping* | *Idle processes waiting for an event* |
| *stopped* | *Suspended (SIGSTOP) processes* |
| *zombie* | *Defunct processes waiting to be cleaned up* |

*A zombie process means the process has finished execution but its parent hasn't collected its exit status.*

**%Cpu(s): 3.2 us, 1.0 sy, 0.0 ni, 95.6 id, 0.0 wa, 0.0 hi, 0.2 si, 0.0 st**

| *Field* | *Meaning* |
|---|---|
| *us* | *User mode time* |
| *sy* | *System (kernel) mode time* |
| *ni* | *Time with adjusted nice priority* |
| *id* | *Idle time* |
| *wa* | *I/O wait (time waiting for disk I/O)* |

*Field Meaning*
*hi Hardware interrupt time*
*si Software interrupt time*
*st Stolen time (time used by hypervisor for another VM)*

**MiB Mem : 7989.7 total, 2567.4 free, 1834.1 used, 3588.2 buff/cache**
**MiB Swap: 2048.0 total, 2048.0 free, 0.0 used. 5332.1 avail Mem**

*Field Meaning*
*total Total physical memory (RAM)*
*free Currently unused memory*
*used Memory actively used by processes*
*buff/cache Memory used for buffers and cache (can be freed if needed)*
*swap Disk space used as virtual memory when RAM is full*
*A common misconception is that high memory usage is bad — Linux uses free memory efficiently for caching.*

| PID | USER | PR | NI | VIRT | RES | SHR | S | %CPU | %MEM | TIME+ | COMMAND |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 1254 | root | 20 | 0 | 700484 | 8236 | 5852 | S | 1.7 | 0.1 | 0:04.76 | systemd |
| 2305 | user1 | 20 | 0 | 159720 | 6508 | 5152 | S | 0.3 | 0.1 | 0:00.92 | bash |
| 2410 | user1 | 20 | 0 | 242348 | 8652 | 7068 | R | 0.3 | 0.1 | 0:00.45 | top |

**Process List (Main Table)**
**Each line represents one process.**
*Column Description*
*PID Process ID*
*USER Process owner*
*PR Priority*
*NI Nice value (used to adjust process priority)*
*VIRT Virtual memory used (includes code + data + shared libraries)*
*RES Resident memory (actual RAM used)*
*SHR Shared memory with other processes*
*S Process state (R=Running, S=Sleeping, T=Stopped, Z=Zombie)*
*%CPU CPU usage percentage*
*%MEM Memory usage percentage*
*TIME+ Total CPU time used by the process*
*COMMAND Name of the command or process*

**Shortcuts**
**q Quit**
**h Help screen**
**k Kill a process (enter PID)**
**r Renice a process (change its priority)**
**P Sort by CPU usage (default)**
**M Sort by memory usage**

| | |
|---|---|
| **T** | **Sort by time** |
| **1** | **Show CPU usage for each core separately** |
| **u** | **Filter processes by username** |
| **Shift + W** | **Save current configuration** |
| **Shift + H** | **Show threads as separate processes** |
| **Shift + F** | **Customize columns displayed** |

***Example 2: Sort by Memory Usage***
**While in top, press:**
**M - Processes will be sorted by memory consumption, highest first.**

***Example 3: Filter by User***
**To see only your processes:**
**u - Then type your username (e.g., student).**

***Example 4: Kill a Process***
**In top:**
**k = Enter the PID and press Enter.You can specify a signal number (e.g., 9 for SIGKILL).**

***Example 5: Show All CPU Cores***
**Press:**
**1 This displays CPU usage for each core — useful on multi-core systems.**

**Common Options (Command-Line)**

| Option | Description |
|---|---|
| **-u <user>** | **Show only processes for a specific user** |
| **-n <num>** | **Display for <num> iterations and exit** |
| **-b** | **Batch mode (non-interactive, for logging)** |
| **-d <delay>** | **Delay (in seconds) between refreshes** |
| **-p <PID>** | **Monitor only specific PID** |
| **-i** | **Ignore idle and zombie processes** |

**Example:**
**top -u student -n 5 -d 2**
**→ Shows processes owned by student, updates every 2 seconds, and runs 5 cycles before exiting.**

**Quick Reference**

| Command | Purpose |
|---|---|
| **top** | **Start real-time monitoring** |
| **top -u user** | **Show processes of a user** |
| **top -n 5 -d 1** | **Run 5 updates, 1s apart** |
| **M** | **Sort by memory usage** |
| **P** | **Sort by CPU usage** |
| **k** | **Kill a process** |
| **1** | **Show all CPU cores** |
| **q** | **Quit top** |

# strace (System Trace)

**The strace command in Linux is used to trace system calls and signals made by a process.**

**It helps in understanding how a program interacts with the operating system kernel — for example, when it opens files, reads input, writes output, or creates new processes.**
**This command is very useful for:**

- **Debugging programs**
- **Learning how system calls work**
- **Finding out why a program fails or behaves unexpectedly**

## Basic Syntax

- `strace [options] command [arguments]`
- **or**
- `strace -p <pid>`

**Every program running on Linux ultimately communicates with the kernel through system calls.**
`strace intercepts and records these calls, showing what the process is requesting from the operating system.`

`1.strace ls`

`This will run the ls command, but instead of showing only the directory listing, it will print all the system calls ls makes — for example:`

```
execve("/bin/ls", ["ls"], 0x7ffd8d0a9a70 /* 52 vars */) = 0
brk(NULL)                               = 0x55eb8f502000
access("/etc/ld.so.preload", R_OK)      = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, ".", O_RDONLY|O_NONBLOCK|O_CLOEXEC|O_DIRECTORY) = 3
...
write(1, "Desktop\nDocuments\nDownloads\n", 29) = 29
```

`Here you can see calls like:`

- `openat() – opening a directory`
- `write() – writing output to the terminal`
- `close() – closing file descriptors`

`2. Trace an Existing Process`

```
strace -p 1234
```

`This attaches strace to an existing process with process ID (PID) 1234 and shows its system calls in real time.`
`Use this carefully — the process will pause briefly while being attached.`

`3. Count System Calls`

```
strace -c ls
```

`This summarizes how many times each system call was made and how much time was spent in each call.`

Example Output:

| % time | seconds | usecs/call | calls | errors | syscall |
|---|---|---|---|---|---|
| 55.12 | 0.000088 | 1 | 88 | | openat |
| 25.33 | 0.000040 | 0 | 72 | | fstat |
| 12.34 | 0.000020 | 0 | 33 | | close |
| 7.21 | 0.000012 | 0 | 12 | | write |
| 100.00 | 0.000160 | | 205 | | total |

This is very useful for teaching how frequently different system calls are made by user-level commands.

4. Trace Only Specific System Calls

```
strace -e open,read,write ls
```

This will show only the open(), read(), and write() system calls — filtering out the rest.

---

5. Trace a Program and Its Child Processes

```
strace -f ./myprogram
```

-f ensures that any processes created using fork() or exec() are also traced.

Commonly Seen System Calls

| System Call | Description |
|---|---|
| open() | Open a file |
| read() | Read from a file descriptor |
| write() | Write to a file descriptor |
| close() | Close a file descriptor |
| execve() | Execute a program |
| fork() | Create a new process |
| mmap() | Map files or devices into memory |
| brk() | Change data segment size (for memory allocation) |

## Command: fuser (File/Filesystem/PortUser Identifier)

---

🔍 Purpose

The fuser command shows which processes are currently using a specific file, directory

It is often used to:

- Identify which process is using a file, mount point, or device
- Detect which process is listening on a network port
- Terminate processes that are using a specific resource

---

🧩 Basic Syntax

```
fuser [options] name
```

Where:

- name → can be a file, directory, mount point, or network port

Example 1: Identify Process Using a File

**fuser test.txt**
**Output:**
**test.txt:  3210**

**🧩 Explanation**

- **3210 → Process ID (PID) of the process using the file test.txt**

**You can then check what that process is:**
**ps -p 3210**

---

**📄 Example 2: Show Processes Using a Directory**
**fuser /home/user/**
**Output:**
**/home/user/:  1203c  2145e  3189r**

**🧩 Explanation of Suffixes (Access Types):**

| Symbol | Meaning |
|---|---|
| c | Current directory |
| e | Executable being run |
| f | Open file |
| r | Root directory |
| m | Memory-mapped file or library |

**So in the example above:**

- **PID 1203 → has /home/user as current directory**
- **PID 2145 → executing something from /home/user**
- **PID 3189 → has /home/user as root directory**

---

**📄 Example 3: Show Processes Using a Mount Point**
**If you have an external drive or partition mounted (say /mnt/usb):**
**sudo fuser -vm /mnt/usb**
**Output:**

| | USER | PID | ACCESS | COMMAND |
|---|---|---|---|---|
| /mnt/usb: | user1 | 421 | ..c.. | bash |
| | user1 | 578 | ..f.. | cat |

**🧩 Explanation:**

- **The -v option gives a verbose table showing username, PID, access type, and command name.**
- **Very useful before unmounting a drive — you can see which process is using it.**

## Example 7: Display Usernames Along with PIDs

- `fuser -u test.txt`
- **Output:**
- `test.txt:  3210(user1)`
- 🧩 Useful when multiple users are logged in and you want to see who is accessing a resource.

## Common Options Summary

| Option | Description |
|---|---|
| `-v` | Verbose output (shows user, PID, access type, and command) |
| `-u` | Show username of process owner |
| `-k` | Kill processes using the resource |

| Option | Description |
|---|---|
| `-i` | Ask for confirmation before killing |
| `-n <space>` | Specify namespace (`file`, `tcp`, `udp`) |
| `-a` | Show all specified files, even if unused |
| `-m` | Name is a mounted file system |
| `-s` | Silent mode (no messages) |

**Sample Observation Table**

| Command | Description | Sample Output |
|---|---|---|
| fuser test.txt | Show PID using file | test.txt: 2145 |
| fuser -v test.txt | Verbose info | USER PID ACCESS COMMAND |
| sudo fuser 22/tcp | Show PID using port 22 | 22/tcp: 1034 |
| sudo fuser -k 8080/tcp | Kill process on port 8080 | (process terminated) |

# Command: time

**The time command is used to measure how long a command or program takes to execute.**
**It reports three key types of time:**

1. **real — Total elapsed (wall clock) time**
2. **user — CPU time spent in user mode**
3. **sys — CPU time spent in kernel mode**

**Basic Syntax**

**time [options] command [arguments]**

**Example:**

**time ls -l**

---

**Example 1: Measuring Execution Time**
**Run: time sleep 3**
**Output:**
**real 0m3.003s**
**user 0m0.000s**
**sys 0m0.001s**

**Explanation:**

| Field | Meaning |
|---|---|
| **real** | **Actual time elapsed (3.003 seconds in this case) — includes waiting time and CPU time** |
| **user** | **Time the CPU spent executing user code (non-kernel mode)** |
| **sys** | **Time the CPU spent executing system (kernel) code on behalf of the process** |

🤔 So here, the command sleep 3 didn’t use CPU, but it took 3 seconds of wall time just waiting.
Example 2: Comparing Commands
Let’s compare two ways to print numbers from 1 to 100000.
Using echo (slow):
time for i in $(seq 1 100000); do echo -n ""; done
Using seq (fast):
time seq 1 100000 > /dev/null
You’ll notice the real time and user/sys times are much smaller for the seq command.
This helps students understand efficiency and CPU usage differences.

# gdb (GNU Debugger)

Purpose
gdb is the GNU Project Debugger, used to debug programs written in languages like C, C++, and Fortran.
It allows you to:

- Run programs step by step
- Examine variables and memory
- Set breakpoints to stop execution at specific points
- Inspect call stacks and understand crashes

---

Basic Syntax
gdb [options] [program] [core]
Typical usage:
gdb ./a.out
Here, a.out is your compiled executable (from a C or C++ program).

---

🏵 Before Using gdb
You must compile your C program with debugging information included:
gcc -g myprogram.c -o myprogram
The -g option tells the compiler to include debugging symbols (function names, variable names, etc.) in the binary.

---

▧ Starting GDB
Start gdb with your program:
gdb ./myprogram
You’ll see:
GNU gdb (GDB) 13.1
Reading symbols from ./myprogram...
(gdb)
Now you’re inside the gdb interactive prompt.

---

🏵 Commonly Used GDB Commands

| Command | Description |
|---|---|
| run | Start the program under gdb |
| break <line_number> | Set a breakpoint at a specific line |
| break <function_name> | Set a breakpoint at a function |
| next | Execute the next line (skips over function calls) |

| Command | Description |
|---|---|
| step | Execute the next line (enters into functions) |
| continue | Continue execution until the next breakpoint |
| print <variable> | Display the value of a variable |
| backtrace | Show the function call stack |
| list | Show source code around the current line |
| info locals | Show all local variables |
| quit | Exit gdb |

**Example: Debugging a Simple Program**

**C Program: buggy.c**

```
#include <stdio.h>
int main() {
   int a = 5, b = 0;
   int c = a / b;  // Division by zero
   printf("Result = %d\n", c);
   return 0;
}
```

***Step 1: Compile with -g***

**gcc -g buggy.c -o buggy**

***Step 2: Run with gdb***

**gdb ./buggy**

***Step 3: Start the Program***

**At the (gdb) prompt, type:**

**run**

**You'll see:**

```
Program received signal SIGFPE, Arithmetic exception.
0x0000000000401134 in main () at buggy.c:5
5       int c = a / b;  // Division by zero
```

**This shows exactly where the program crashed and why.**

***Step 4: Inspect Variables***

```
(gdb) print a
$1 = 5
(gdb) print b
$2 = 0
```

**You can see that the division by zero occurred because b = 0.**

***Step 5: List Source Around Current Line***

```
(gdb) list
```

---

**Step 6: Quit** `(gdb) quit`

**strings**

**Purpose**

**The strings command is used to extract and display readable text (ASCII or Unicode strings) from binary files, such as executables, object files, or core dumps.**

**It's very useful to:**

- **Find hidden text or messages inside compiled programs or data files**
- **Inspect binaries for debugging or reverse engineering**
- **Check what libraries, error messages, or paths a binary uses**

---

**Basic Syntax**

**strings [options] filename**
**If used without options, strings prints all readable text sequences of 4 or more characters found in the file.**

**Example 1: Using strings on an Executable**
**Suppose you have a compiled C program:**
**gcc -o hello hello.c**
**Now run:**
**strings hello**
**Example Output:**
**/lib64/ld-linux-x86-64.so.2**
**libc.so.6**
**Hello, world!**
**printf**
**__libc_start_main**
**GLIBC_2.2.5**
**🧩 Explanation:**

- **It shows text strings inside the binary file.**
- **You can see library names (libc.so.6), system paths, and even your program's output string ("Hello, world!").**

**Useful Options Summary**

| Option | Description |
|---|---|
| -a | Scan the entire file (default) |
| -n <number> | Minimum string length (default = 4) |
| -t <format> | Print offset (format: d=decimal, o=octal, x=hex) |
| -e <encoding> | Specify character encoding (e.g., s=single-byte, l=16-bit little-endian) |
| -f | Print the filename before each string (useful with multiple files) |

# objdump (Object Dump)

**Purpose**
**The objdump command in Linux displays detailed information about object files and executables.**
**It allows you to inspect:**

- **Machine code (assembly) generated by the compiler**
- **Headers and sections of binary files**
- **Symbol tables and debugging information**

**It's extremely useful for learning:**

- **What happens after compilation**

- **How the assembler and linker organize executables**
- **How to read assembly code and understand binary structure**

---

**Basic Syntax**
**objdump [options] filename**
**Example:**
**objdump -d a.out**

---

**🧠 What It Works On**
- **.o files (object files produced by the compiler before linking)**
- **Executables (like a.out or your compiled programs)**
- **Static libraries (.a)**
- **Dynamic libraries (.so)**

---

**1. Disassemble an Executable**
**objdump -d a.out**
**This shows the assembly code generated from your C source after compilation.**
**Example Output:**
**a.out: file format elf64-x86-64**

**Disassembly of section .text:**

**0000000000001139 <main>:**
**1139: 55 push %rbp**
**113a: 48 89 e5 mov %rsp,%rbp**
**113d: b8 00 00 00 00 mov $0x0,%eax**
**1142: 5d pop %rbp**
**1143: c3 ret**
**🧩 Explanation:**
- **The section .text contains the program's machine instructions.**
- **Each line shows:**
  - **Address in memory**
  - **Hex code (machine code)**
  - **Assembly instruction**

**This helps students visualize how C code is translated into assembly.**

**Commonly Used Options Summary**

| Option | Description |
|---|---|
| -d | Disassemble executable code |
| -S | Show source code along with assembly |
| -t | Show symbol table |
| -h | Show section headers |
| -x | Show all headers |
| -f | Show file format info |
| --disassemble=function | Disassemble only a given function |

**To study the use of objdump command for analyzing executable files.**
**Procedure:**
1. **Write and compile a simple program:**

2. **#include <stdio.h>**
3. **int main() {**
4. **printf("Operating Systems Lab\n");**
5. **return 0;**
6. **}**
7. **Compile with debug symbols:**
8. **gcc -g hello.c -o hello**
9. **Use these commands and observe the outputs:**
10. **objdump -f hello**
11. **objdump -h hello**
12. **objdump -t hello**
13. **objdump -S hello**
14. **Note the section names, symbol addresses, and assembly instructions.**

# nm (Name List)

**Purpose**
The nm command lists **symbols** (functions, variables, etc.) from **object files**, **static libraries**, and **executables**.
It's very useful to:

- Inspect **which symbols (functions/variables)** are defined or undefined in a file.
- Understand **linker errors** like "undefined reference".
- Study how the compiler organizes symbols in **different sections** (.text, .data, .bss).

---

**Basic Syntax**
nm [options] filename
Example:
nm a.out

---

**What are Symbols?**
Symbols represent **identifiers** (names) in your program — for example:

- Functions (like main, printf)
- Global/static variables
- External references (things defined in another file)

Each symbol has:

1. **Address** (where it's stored in memory)
2. **Type** (function, variable, etc.)
3. **Name**

---

**Example 1: Simple Program**
**sum.c**

```
#include <stdio.h>

int a = 5;        // global variable
int b;            // uninitialized global variable

int sum(int x, int y) {
   return x + y;
```

```
}

int main() {
    int c = sum(a, 10);
    printf("%d\n", c);
    return 0;
}
Compile it:
gcc -c sum.c     # creates object file sum.o
Then run:
nm sum.o
```

---

**Sample Output:**

```
0000000000000000 T sum
0000000000000010 T main
0000000000000004 D a
0000000000000008 B b
                 U printf
```

---

**Understanding the Columns**

| Column | Meaning |
|---|---|
| Address | Memory address (if known) |
| Type | Symbol type (a single letter) |
| Name | Symbol name |

---

**Symbol Type Codes**

| Code | Section | Meaning |
|---|---|---|
| **T** | .text | Function defined in this file |
| **t** | .text | Local (static) function |
| **D** | .data | Initialized global variable |
| **d** | .data | Initialized static variable |
| **B** | .bss | Uninitialized global variable |
| **b** | .bss | Uninitialized static variable |
| **U** | – | Undefined (used but not defined here — will be resolved by linker) |
| **C** | – | Common (uninitialized data to be allocated by the linker) |
| **R** | .rodata | Read-only data (constants, strings) |
| **W** | – | Weak symbol |
| **?** | – | Unknown type |

**To study the use of nm command for listing symbol information in object and executable files.**

**Procedure:**

1. **Write and compile a simple program:**
2. **#include <stdio.h>**
3. 
4. **int x = 10;**
5. **int y;**
6. 
7. **int square(int n) {**
8. **    return n * n;**

```
}

int main() {
    printf("%d\n", square(x));
    return 0;
}
```

15. Compile it to object form:
16. gcc -c test.c
17. Run:
18. nm test.o
19. Observe the symbol table output and identify:
    - Defined functions
    - Initialized/uninitialized variables
    - Undefined external symbols

# Command: file

---

**🔍 Purpose**

**The file command is used to identify the type of a file.**
**It examines the file's contents, not its extension, and tells you what kind of data it holds.**
**This helps you distinguish between:**

- **Text files**
- **Executable files**
- **Object files**
- **Scripts**
- **Images, PDFs, etc.**

---

**🛠 Basic Syntax**

**file [options] filename**
**Example:**
**file a.out**

---

**⚙ How It Works**

**The file command uses a three-step test to determine a file's type:**

1. **Filesystem test – Checks the type from filesystem information (e.g., directory, symbolic link, block device).**
2. **Magic test – Reads the first few bytes of the file (called the magic number) and compares them with entries in /usr/share/misc/magic.mgc.**
3. **Language test – If it's a text file, file checks whether it contains source code (like C, Python, shell script, etc.).**

---

**Example 1: Identifying Common Files**
**Let's try on different files.**
**file /bin/ls**
**Output:**
**/bin/ls: ELF 64-bit LSB executable, x86-64, dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2**

**🛠 Explanation:**

- **ELF → Executable and Linkable Format (used for binaries in Linux)**
- **64-bit → Architecture type**

- **Dynamically linked → Uses shared libraries**
- **Interpreter → Loader that runs the binary**

---

**file hello.c**
**Output:**
**hello.c: C source, ASCII text**

---

**file test.txt**
**Output:**
**test.txt: ASCII text**

---

**file image.png**
**Output:**
**image.png: PNG image data, 800 x 600, 8-bit/color RGBA**

---

**file script.sh**
**Output:**
**script.sh: Bourne-Again shell script, ASCII text executable**

# Command: od (Octal Dump)

---

**🔍 Purpose**
**The od command is used to display file contents in various numeric formats, such as:**

- **Octal (default)**
- **Hexadecimal**
- **Decimal**
- **Character**
- **ASCII**

**It allows you to view raw binary data — the exact bytes stored in memory or a file — which is extremely useful in:**

- **Debugging programs**
- **Examining binary or data files**
- **Understanding file formats**
- **Learning how text and numbers are represented internally**

---

**Basic Syntax**
**od [options] filename**
**Example:**
**od myfile.txt**

---

**🧠 Default Behavior**
**By default, od displays the file's contents in octal format.**
**Example:**
**Create a simple file:**
**echo "OS" > test.txt**
**Run:**
**od test.txt**
**Output:**
**0000000 117 123 012**

**0000003**
**Explanation:**

- **The first column (0000000, 0000003) → byte offset in the file**
- **The numbers (117 123 012) → ASCII values of characters in octal**
  - **117 = O**
  - **123 = S**
  - **012 = newline (\n)**

---

**💠 Viewing in Different Formats**
**You can change how the bytes are displayed using options.**

---

**◇ 1. Hexadecimal Output**
**od -x test.txt**
**Output:**
**0000000 534f 000a**
**0000003**
**💠 Explanation:**

- **534f = Hex representation of "OS" (O=4F, S=53)**
- **000a = newline (\n)**

---

**◇ 2. Character Output**
**od -c test.txt**
**Output:**
**0000000 O S \n**
**0000003**
**💠 Explanation:**
**Displays the character equivalents of bytes — much easier to read for text files.**

---

**◇ 3. ASCII + Octal Together**
**od -a test.txt**
**Output:**
**0000000 O S nl**
**0000003**
**💠 Explanation:**

- **Shows ASCII names instead of characters (e.g., nl for newline).**

---

**◇ 4. Decimal Output**
**od -i test.txt**
**Output:**
**0000000 12335 00010**
**0000003**
**💠 Displays integer (decimal) representation of the bytes.**

---

**◇ 5. Binary Output**
**od -b test.txt**
**Output:**
**0000000 117 123 012**
**0000003**
**💠 Same as default — octal representation of bytes.**

---

**◇ 6. Hex Dump (Most Common Use)**

**od -t x1 test.txt**

**Output:**

```
0000000 4f 53 0a
0000003
```

**💮 Explanation:**

- **4f → O**
- **53 → S**
- **0a → newline**
  **Shows each byte in hexadecimal (x1 = one-byte units).**

---

**◇ 7. Combine Character and Hex**

**od -tx1 -c test.txt**

**Output:**

```
0000000  4f  53  0a
         O   S  \n
0000003
```

**💮 Displays both — useful for teaching how ASCII characters map to byte values.**

---

**Viewing Specific Number of Bytes**

**Use the -N option:**

**od -c -N 5 myfile.txt**

**💮 Displays only the first 5 bytes.**

---

**To study the use of the od command for displaying file contents in octal, hexadecimal, and character formats.**

---

**Procedure:**

1. **Create a text file:**
2. **echo "Operating Systems" > sample.txt**
3. **Display in different formats:**
4. **od sample.txt**
5. **od -c sample.txt**
6. **od -x sample.txt**
7. **od -t x1 sample.txt**
8. **od -a sample.txt**
9. **Observe and record differences in the output.**

---

**Sample Observation Table**

| Command | Description | Output (example) |
|---|---|---|
| od sample.txt | Octal representation | 117 160 145 ... |
| od -c sample.txt | Character view | O p e r ... |
| od -x sample.txt | Hexadecimal view | 704f 6572 ... |
| od -t x1 sample.txt | One byte per hex value | 4f 70 65 ... |
| od -a sample.txt | ASCII names | O p e r ... |

# Command: xxd (Hex Dump Tool)

---

## 🔍 Purpose

**The xxd command creates a hexadecimal (hex) dump of a file or standard input.**
**It is used to:**

- **Display the binary contents of a file in human-readable hex format**
- **Show the corresponding ASCII characters**
- **Recreate the original file from the hex dump (reverse operation)**

**This makes it extremely useful for:**

- **Studying how data is stored in files**
- **Debugging binary data**
- **Understanding file formats**
- **Working with encrypted or compiled data**

---

## ⚙️ Basic Syntax

**xxd [options] filename**
**Example:**
**xxd sample.txt**

---

## 📄 Example 1: Simple Text File

**Create a file:**
**echo "OS Lab" > sample.txt**
**Now run:**
**xxd sample.txt**
**Output:**
**00000000: 4f53 204c 6162 0a OS Lab.**

## ⚙️ Explanation

| Column | Meaning |
|---|---|
| 00000000 | Byte offset (address) |
| 4f53 204c 6162 0a | Hexadecimal representation of file content |
| OS Lab. | ASCII characters (the rightmost column) |

**Each pair of hex digits (like 4F) represents one byte.**
**So:**

- **4F → O**
- **53 → S**
- **20 → space**
- **4C → L**
- **61 → a**
- **62 → b**
- **0A → newline**

---

## 📄 Example 2: Display in Hex Only

**If you want only the hex output (no ASCII):**
**xxd -p sample.txt**
**Output:**
**4f53204c61620a**
**⚙️ This gives a plain hex stream, often used for data transmission or comparison.**

---

## 📄 Example 3: Reverse a Hex Dump

**xxd can recreate the original file from its hex dump.**

1. **Create a hex dump and save it:**
2. **xxd sample.txt > dump.hex**
3. **Recreate the original file:**

4. **xxd -r dump.hex newfile.txt**
5. **Check:**
6. **cat newfile.txt**

**Output:**
**OS Lab**
☑ **This round-trip conversion makes xxd an excellent teaching tool for showing how text translates to binary and back.**

---

▧ **Example 4: Specify Bytes per Line**
**By default, xxd shows 16 bytes per line.**
**To change that:**
**xxd -c 8 sample.txt**
**Output:**
**00000000: 4f53 204c 6162 0a OS Lab.**
🏵 **Shows 8 bytes per line instead of 16 — useful for short files.**

**Common Options Summary**

| Option | Description |
|---|---|
| -p | Plain hex dump (no offsets or ASCII) |
| -r | Reverse operation (hex → binary) |
| -c <n> | Show *n* bytes per line |
| -s <n> | Skip *n* bytes from start |
| -l <n> | Limit to *n* bytes |
| -A | Display addresses in decimal |
| -g <n> | Group bytes (1, 2, or 4) |
| -e | Switch to little-endian mode for multibyte groups |

---

🖉 **Simple Lab Exercise**
**Aim:**
**To study the use of the xxd command for viewing and converting file contents in hexadecimal format.**

---

**Procedure:**
1. **Create a sample text file:**
2. **echo "Operating Systems Lab" > oslab.txt**
3. **Generate hex dump:**
4. **xxd oslab.txt**
5. **View plain hex:**
6. **xxd -p oslab.txt**
7. **Reverse the hex dump:**
8. **xxd oslab.txt > dump.hex**
9. **xxd -r dump.hex new.txt**
10. **cat new.txt**
11. **Observe and compare the outputs.**

---

**Sample Observation Table**

| Command | Description | Sample Output |
|---|---|---|
| xxd oslab.txt | Hex + ASCII dump | 4f53 204c 6162 → OS Lab |
| xxd -p oslab.txt | Plain hex | 4f53204c6162 |
| xxd -r dump.hex new.txt | Reverse hex dump | Text restored |
| xxd -c 8 oslab.txt | 8 bytes per line | Neater view |